G-TRaMS: Gasan Tricycle Franchise Records and Monitoring System

Keywords: *Tricycle, Franchise, REST API, MERN*

Proponent(s): 1. *Jhon Kyn Axix H. Cabrigas - Lead Backend Developer* 2. *Justine S. Lachica - Frontend Developer* 3. *Jay Vincent G. Motol - QA 4. Ian Simon Francis M. Rey - DevOps*

CHAPTER 1: TECHNICAL DESCRIPTION & PROPOSAL

1.1 Background of the Study

Job Order personnel in the Office of the Vice Mayor of Gasan tasked with managing tricycle franchises are having a difficult time due to record-keeping problems. They also experience inefficient data retrieval. Likewise, this makes tracking applicant statuses problematic. The franchise process today is slow and inefficient as it relies on physical documents and Microsoft Excel for manual record-keeping.

1.2 Objectives (General and Specific)

Your objectives must follow the SMART criteria (Specific, Measurable, Achievable, Relevant, and Time-bound).

- General Objective: *To design, develop, and deploy a web-based Tricycle Franchise Records and Management System using the MERN stack for the Job Order Staffs of the Office of the Municipal Vice Mayor of Gasan that processes tricycle franchise of tricycle operators.*
- Specific Objectives:

  1. To achieve a secure User Authentication module using JWT and bcryptjs, separating access between 'Applicant/User' and 'Admin' roles.

  2. To create a RESTful API backend that processes dynamic CRUD operations for at least 4 distinct database collections (User, Franchise, Reports, Calendar)

  3. To assimilate a React.js frontend that oversee the processes of tricycle franchise like approved, denied, and canceled applications and displays it on the dashboard.

  4. To utilize the fully functional database, server, and client applications to cloud hosting platforms (MongoDB Atlas, and Vercel) with zero critical build errors.

1.3 Review of Related Literature and Systems

- *Related System 1 (Development of a Web-based data Management System (ERMS)) — This system improves record classification, storage, controlled access, and retrieval for a Philippine government office, which is directly applicable to franchise data handling. Its shortcoming is that it was created for a different office workflow, hence it does not cover tricycle-franchise status tracking or MERN-based deployment.*

- *Related System 2 (Design and Implementation of a Web-based Document Management System) - By incorporating digital entry, search, recent records, and internal routing, this system addressed manual workflow and delayed document search. Nevertheless, it did not prioritize applicant role separation or contemporary RESTful API integration, and it employed a more basic PHP/MySQL configuration.*

- *Related System 3 (A Secured Cloud-Based Electronic Document Management System) - This system added safe document entry, retrieval, and sharing with encrypted storage to overcome paper-based inefficiencies. Its focus on document management rather than franchise application approvals, denials, and cancellations is one of its limitations.*

- *Related System 4 (MERN Stack Web-Based Education Management Information System) - demonstrated how MERN may facilitate web-based records for centralized data management and improved decision-making. The fact that it was created for educational data rather than municipal franchise processing or applicant dashboards is one of its limitations.*

- *Related System 5 (Postgraduate Student Management System) - This React.js and Express.js system enhanced stakeholder coordination and coordinated multi-role workflows. Its inability to handle local government record-keeping and tricycle franchise monitoring stems from the fact that it was created for academic administration.*

- *Related System 6 (Streamlining Tax and Administrative Document Processing) - To cut down on manual processing and boost productivity, this system automates administrative document handling. Its document-centric design and lack of the applicant/admin job structure required in a franchise management system are its main drawbacks.*

- *Related System 7 (Scalable Electronic Health Record Management System) - This system demonstrated how organized digital records can enhance consistency and scalability in sensitive record contexts. The fact that it is designed for healthcare data rather for municipal franchise applications and status tracking is one of its limitations.*

- *Related System 8 (Administrative Dashboard for Web-Based Program Monitoring) - This system employed a dashboard to track user activity and progress in a single, integrated view. Its shortcoming is that it was designed for behavioral therapies rather than government records retrieval or franchise approval workflows.*

- *Related System 9 (Community Vibrancy Dashboard) - By offering prompt visual feedback and progress monitoring via an online dashboard, this solution enhanced decision-making. Its concentration on community indicators rather than applicant-level processing or record keeping is one of its limitations.*

- *Related System 10 (Enhancing JWT Authentication and Authorization in Web Applications) - This study used JWT-based authentication in conjunction with user-behavior data to increase login confidence. The approach's drawback is that it is not a comprehensive franchise management workflow system, but it does include security logic.*

- *Related System 11 (Secure Authentication in E-Government 2.0) - For safe online access in government systems, this essay contrasted session-based and JWT-based authentication. Its primary focus on authentication techniques rather than the whole CRUD and dashboard features your project need is one of its limitations.*

- *Related System 12 (Work Management Tool Using MERN Stack) - This system included task creation, assignment, tracking, and role-based access control using MongoDB, Express.js, React, and Node.js. Its drawback is that it was not intended for processing applicant status in a municipal office, but rather for job management.*

- *Related System 13 (Web-Based Attendance Management System) - This system offered real-time analytics and replaced manual tracking with React.js, Node.js/Express.js, and RBAC. The fact that it was designed for attendance records rather than franchise application lifecycle management is one of its limitations.*

- *Related System 14 (Visual Interface Workflow Management System) - This system integrated dashboards that display task status in a visual workflow with MongoDB, Node.js/Express.js, and React.js. Its focus on project monitoring*

rather than government franchise records and applicant approvals is one of its limitations.

- *Related System 15 (MongoDB: Meeting the Dynamic Needs of Modern Applications) - The usage of MongoDB for scalable and effective data management in contemporary web systems is supported by this evaluation. The fact that it is a database-focused review rather than a full application with dashboards, franchise workflows, or authentication is one of its limitations.*

1.4 Methodology & Solution

To solve the problem of record keeping and data storing and retrieval, this project will use the MERN stack (MongoDB, Express.js, React.js, and Node.js) to develop a web-based Tricycle Franchise Records and Management System following a client-server architecture. The backend will provide RESTful APIs for handling CRUD operations on Admin, User, and Franchise data, with secure authentication using JWT and bcryptjs for role-based access control. The React.js frontend will allow applicants to submit and track applications, while administrators can manage and process franchise requests efficiently. MongoDB Atlas will be used for cloud-based data storage, and system integration will be achieved through asynchronous API communication. The system will undergo unit, integration, and end-to-end testing before deployment to ensure reliability, scalability, and improved efficiency.

1.5 Target Users / Beneficiaries

- Beneficiary 1 (Tricycle Operators): The tricycle operators and drivers in the Municipality of Gasan will use the system to submit their franchise applications, upload necessary documents like OR/CR and Cedula, track their application status in real-time, and submit support tickets or inquiries without the need to physically visit the municipal office.
- Beneficiary 2 (System Admins / Municipal Staff): The Job Order personnel and designated staff in the Office of the Municipal Vice Mayor of Gasan hold administrative powers. They can approve, deny, or cancel franchise applications, view the masterlist of all registered tricycles, generate analytical reports, manage user accounts, and update the office calendar schedule.

1.6 Significance of the Study

The implementation of the G-TRAMS project brings significant value to the local government of Gasan by modernizing an otherwise slow, paper-based process. By shifting from manual Microsoft Excel records and physical folders to a centralized, cloud-based database, the LGU can eliminate data loss and redundancy. It significantly reduces the processing time for both new applications and renewals, saving time and transportation costs for the tricycle operators. Furthermore, the built-in analytics dashboard provides the municipal officials with accurate,

real-time data regarding active units per TODA, which is critical for local transport policy-making and urban planning.

CHAPTER 2: MINIMUM VIABLE PRODUCT (MVP) DOCUMENTATION

2.1 System Architecture

G-TRAMS utilizes a modern Client-Server Architecture via the MERN stack. The frontend is built with React.js and is hosted on Vercel, serving as the user interface for both operators and admins. It communicates asynchronously via HTTP requests to the Node.js/Express.js backend hosted on Render. The backend acts as the bridge, processing business logic, handling JWT authentication, and securely connecting to a MongoDB Atlas cloud database to store and retrieve records. Uploaded files, such as profile pictures and OR/CR documents, are processed by Multer and securely stored using Cloudinary.

2.2 Tech Stack & Tools

- Frontend: React.js, Vite, React Router DOM, Recharts (for analytics visualization)
- Backend: Node.js, Express.js, JSON Web Tokens (JWT), bcryptjs, Multer, Cloudinary, Nodemailer
- Database: MongoDB Atlas, Mongoose
- Testing: Postman
- Deployment: Vercel / Render

2.3 Database Schema (Data Modeling)

Collection 1 (Users): Stores account information.

Fields: name (String), email (String, unique), password (String, hashed), role (String: 'admin' or 'operator'), address (String), profilePic (String, Cloudinary URL), otp (String, for password reset).

Collection 2 (Franchises): Stores tricycle and application data. References the User collection.

Fields: operator (ObjectId, ref: 'User'), plateNo (String), todaName (String), zone (String), make (String), made (String), motorNo (String), chassisNo (String), applicationType (String: 'New' or 'Renewal'), status (String: 'Pending', 'Active', 'Expired', 'Cancelled'), orCrUrl (String, Cloudinary URL), cedulaSerialNo (String), cancelReason (String).

Collection 3 (Reports): Stores feedback and help desk tickets.

Fields: operator (ObjectId, ref: 'User'), subject (String), message (String), response (String), status (String: 'Pending' or 'Resolved').

Collection 4 (Calendar): Stores office schedules.

- Fields: date (String), status (String), note (String).

CHAPTER 3: REST API DESIGN DOCUMENTATION

3.1 Overview & Intent

- Base URL: [https://g-trams-web2.onrender.com/api/v1] (https://g-trams-web2.onrender.com/api/v1)
- Purpose: *This API handles the core functionalities of the G-TRAMS system, including user authentication, submission and modification of franchise applications, fetching analytics data, and managing support tickets.*

3.2 Authentication & Authorization

- Strategy: JSON Web Tokens (JWT) passed via the Authorization: Bearer <token> header.
- Roles: 'admin' has full access to all endpoints, including verifying, approving, and deleting records. 'operator' has restricted access, only able to view, create, and update their own franchise applications and reports.

3.3 Endpoint Documentation

(Replicate this block for EVERY route in your system)

- Endpoint Name: Submit New Franchise Application
- Method & URL: [POST] /franchises
- Authorization Required: Yes (Admin/Operator)
- Request Format (Headers & Body):

```
Content-Type: multipart/form-data
Body: todaName, zone, plateNo, make, made, motorNo, chassisNo, orCrDocument (file)
Success Response (201 Created):
{
  "_id": "64b1c2...",
  "plateNo": "ABC 123",
  "status": "Pending",
  "applicationType": "New"
}

Endpoint Name: Update Franchise Status
```

```
Method & URL: PUT /franchises/:id/status
Authorization Required: Yes (Admin only)
Request Format:
Content-Type: application/json
{
  "status": "Active",
  "eSigned": true,
  "releaseDate": "2026-04-20"
}
Success Response (200 OK):
{
  "message": "Franchise status updated successfully."
}
```

3.4 Advanced API Features

- File Handling & Cloud Storage: The API utilizes Multer for intercepting multipart/form-data requests and uploads binary files (images/PDFs) directly to Cloudinary, saving only the secure URL string to the MongoDB database.
- Password Recovery Protocol: Integrated with Nodemailer to securely generate and send time-bound One-Time Passwords (OTP) to user emails for password resets.

3.5 Testing Tools & Examples

- Tools Used: Used: Postman for manual testing of API endpoints and checking HTTP status codes.
- Sample cURL/Fetch Example:

```
// example frontend fetch request for getting masterlist
fetch('[https://g-trams-web2.onrender.com/api/v1/franchises](https://g-trams-web2.onrender.com/api/v1/franchises)',
{
  method: 'GET',
  headers: {
    'Authorization': `Bearer ${token}`
  }
})
```

CHAPTER 4: DEPLOYMENT & LINKS

4.1 Live Application Links

- Frontend Web Application URL: https://g-trams-web2.vercel.app
- Backend API Base URL: https://g-trams-web2.onrender.com
- GitHub Master Repository URL: https://github.com/justineeeelachicaaaa/g-trams-web2.git

4.2 Testing Coverage Summary

- Unit & Integration Tests: The system components were manually tested throughout the development lifecycle using local environments to ensure data flow between React state and Express controllers functions as intended.
- End-to-End (E2E) Tests: Core workflows such as user registration, franchise application submission with file uploads, admin approval, and automated permit generation were verified sequentially to mimic actual user operations.

APPENDICES

- Appendix A: Glossary of API Terms
  - *API (Application Programming Interface) — A set of rules that allows the frontend and backend of the system to communicate with each other.*
  - *Endpoint — A specific URL in the backend where a request is sent, such as /franchises or /users/login.*
  - *Payload — The data sent inside a request or response. For example, the form data submitted when applying for a franchise.*
  - *Token — A secure string used to identify a logged-in user and verify access to protected routes.*
  - *JWT (JSON Web Token) — A token format used for secure authentication and authorization in the system.*
  - *HTTP Request — A message sent from the client to the server to perform an action such as create, read, update, or delete data.*
  - *HTTP Response — The server's reply to a request, containing status codes, messages, or data.*
  - *CRUD — Create, Read, Update, and Delete; the four basic operations performed on database records.*
  - *Middleware — A function that processes requests before they reach the main route handler, often used for authentication, file upload, and error handling.*
  - *Bearer Token — A JWT sent in the Authorization header using the format Bearer <token>.*
  - *RESTful API — An API design style that uses standard HTTP methods like GET, POST, PUT, and DELETE for data operations.*
  - *Multipart/Form-Data — A request format used when submitting files together with text fields, such as OR/CR documents.*
  - *Cloud Storage — Online storage used for files such as uploaded documents and profile pictures, like Cloudinary in the system.*
  - *Dashboard — The main interface that shows summaries, status updates, and analytics for admins and operators.*
- Appendix B: Acronyms & Abbreviations

Acronym	**Meaning**
API	Application Programming Interface
CRUD	Create, Read, Update, Delete

JWT	JSON Web Token
RBAC	Role-Based Access Control
MERN	MongoDB, Express.js, React.js, Node.js
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
UI	User Interface
UX	User Experience
DB	Database
LGU	Local Government Unit
TODA	Tricycle Operators and Drivers Association
OR	Official Receipt
CR	Certificate of Registration
OTP	One-Time Password
JSON	JavaScript Object Notation
PDF	Portable Document Format
SQL	Structured Query Language
No.	Number

- Appendix C: Changelog / Document Revisions

Date	**Revision / Change Description**	**Author**
March 29, 2026	Research phase and initial drafting of the Project Proposal.	Ian Simon Francis Rey
March 29, 2026	Database Schema design and finalizing the system's Technical Description.	Ian Simon Francis Rey
March 30, 2026	**Official Start of Development:** Setting up the MERN stack environment and Backend API architecture.	Justine Lachica

Date	Revision / Change Description	Author
April 1–5, 2026	Development of the Login Module and secure User Authentication using JWT and bcryptjs.	Jhon Kyn Axix Cabrigas
April 6–10, 2026	Rapid Frontend Development: Creating the Admin and Operator dashboards, and integrating core franchise logic.	Justine Lachica
April 11, 2026	Integration of multimedia handling for OR/CR documents via Cloudinary and final UI polishing.	Jay Vincent Motol
April 12, 2026	System testing, bug fixing (API handling), and deployment to Vercel and Render.	Jay Vincent Motol
April 13, 2026	Final Documentation refinement and preparation for project defense.	Ian Simon Francis Rey

- Appendix D: Proof of Validity / Certification *(Attach a scanned copy or screenshot of the validation form/letter from the expert or agency concerned, as required in Section 1.5)*

REFERENCES

- A. B. Author. 2023. Development of a Web-based Records Management System (ERMS) for the Office of Senior Citizen Affairs in the Philippines. ResearchGate. Retrieved from https://www.researchgate.net/publication/388177309

- M. A. Khan, S. Rahman, and T. Islam. 2023. Design and Implementation of a Web-based Document Management System. International Journal of Information Technology and Computer Science. Retrieved from https://www.mecs-press.org/ijitcs/ijitcs-v15-n2/IJITCS-V15-N2-4.pdf

- J. D. Cruz and R. M. Santos. 2023. A Secured Cloud-Based Electronic Document Management System. ResearchGate. Retrieved from https://www.researchgate.net/publication/369768638

- S. Kumar and P. Singh. 2022. MERN Stack Web-Based Education Management Information Systems for Pacific Island Countries. ResearchGate. Retrieved from https://www.researchgate.net/publication/365841553

- L. Garcia and M. Torres. 2024. Postgraduate Student Management System: A Web Application using React.js and Express.js. ResearchGate. Retrieved from https://www.researchgate.net/publication/377443940

- F. Rossi, G. Bianchi, and L. Romano. 2024. Streamlining Tax and Administrative Document Processing through Digital Systems. Information 15, 8. https://doi.org/10.3390/info15080461

- H. Zhang, Y. Liu, and X. Chen. 2023. Scalable Electronic Health Record Management System for Healthcare Data. Systems 11, 7. https://doi.org/10.3390/systems11070346

- R. Smith and K. Johnson. 2024. Administrative Dashboard for Monitoring Web-Based Programs. ScienceDirect. Retrieved from https://www.sciencedirect.com/science/article/pii/S2561326X24000581

- D. Brown and E. Wilson. 2022. Community Vibrancy Dashboard: Enhancing Decision-Making through Data Visualization. ScienceDirect. Retrieved from https://www.sciencedirect.com/science/article/pii/S0380133022001757

- A. Patel and S. Mehta. 2023. Enhancing JWT Authentication and Authorization in Web Applications Based on User Behavior History. ResearchGate. Retrieved from https://www.researchgate.net/publication/370020184

- M. Al-Shehri and A. Alotaibi. 2022. Secure Authentication in E-Government 2.0: A Comparative Study of Session-Based and JWT-Based Approaches. ISG Journal. Retrieved from https://isg-journal.com/isjea/article/view/884

- P. Verma and R. Gupta. 2025. Work Management Tool Using MERN Stack. International Journal of Research Publication and Reviews. Retrieved from https://ijrpr.com/uploads/V6ISSUE4/IJRPR42271.pdf

- S. Das and A. Nair. 2024. Web-Based Attendance Management System Using MERN Stack with Role-Based Access Control. IJIRT. Retrieved from https://ijirt.org/publishedpaper/IJIRT174142_PAPER.pdf

- K. Lee and J. Park. 2025. Visual Interface Workflow Management System for Data Integrity and Project Tracking. ResearchGate. Retrieved from https://www.researchgate.net/publication/399434115

- M. Stonebraker and A. Gupta. 2022. MongoDB: Meeting the Dynamic Needs of Modern Applications. Data 4, 4. https://doi.org/10.3390/data4040093